

Jacobian Space from Scratch: Qwen3.6-27B

Anthropic 的 **Jacobian lens** 主張:語言模型的中間層裡有一個 workspace, 放著還沒說出口的想法,而且讀得出來、也改得動。

這份 notebook 用 MLX 在 Apple Silicon 上把方法從零實作一次,再把論文的主要實驗 搬到 Qwen3.6-27B-4bit 上跑,看哪些站得住、哪些在這顆 model 上不成立。

1. Qwen3.6-27B 的架構,走一次 forward
2. **logit lens**:最便宜的讀法,以及它哪裡不夠
3. **Jacobian lens**:把 logit lens 當成 I 的那一項真的算出來
4. 跨語言:中文問句,英文答案先浮出來
5. **J-space** 本體:稀疏非負分解,以及它佔了多少
6. 換掉 J-space 裡的概念
7. 論文的五個 workspace 面向,一個一個測

模型放在 `models/Qwen3.6-27B-4bit` ([mlx-community 版](#)), 跑在 mlx-vlm 0.6.4。

```

In [1]: from mlx_vlm import load
        from rich import print

def patch_qwen3_5():
    """mlx-vlm #1548: 0.6.4 misses the +1.0 shift on qwen3_5 RMSNorm
    weights,
    which garbles every output. Must run before load()."""
    import mlx_vlm.models.qwen3_5.qwen3_5 as _q
    keys = (".input_layernorm.weight", ".post_attention_layernorm.weight",
            "model.norm.weight", ".q_norm.weight", ".k_norm.weight")
    if getattr(_q.Model, "sanitize", "_patched", False):
        return
    base = _q.sanitize_key
    def sanitize(self, weights):
        shift = any("mtp." in k for k in weights) or any(
            "conv1d.weight" in k and v.shape[-1] != 1 for k, v in
weights.items())
        weights = {k: v for k, v in weights.items() if "mtp." not in k}
        if self.config.text_config.tie_word_embeddings:
            weights.pop("lm_head.weight", None)
        out = {}
        for k, v in weights.items():
            k = base(k)
            if "conv1d.weight" in k and v.shape[-1] != 1:
                v = v.moveaxis(2, 1)
            if shift and any(k.endswith(s) for s in keys) and v.ndim == 1:
                v = v + 1.0
            out[k] = v
        return out
    sanitize._patched = True
    _q.Model.sanitize = sanitize

patch_qwen3_5()

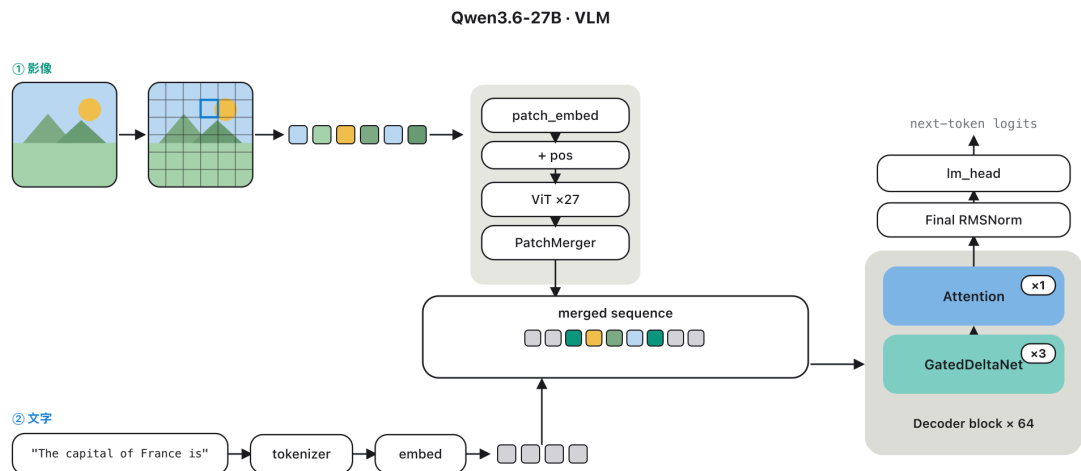
```

```

/Users/happydog/dev/jspace-repro/.venv/lib/python3.13/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user\_install.html
    from .autonotebook import tqdm as notebook_tqdm

```

1. Qwen3.6-27B 的架構



這是多模態模型,圖和文字各走一條前處理,最後併成同一串 sequence 送進 decoder。

文字:tokenizer 把字串切成 token id,再去 embedding 表查出 5120 維向量。

影像:切成 16x16 的 patch,每個 patch 過 vision encoder (ViT) 編碼,再由 PatchMerger 把 2x2 個 patch 併成一個、投影到 5120 維。Qwen 的 vision encoder 設計上接近 SigLIP-2 那一系。

兩邊出來的向量都是 5120 維,併成同一串 sequence 之後,decoder 就不分誰是圖誰是字了。最後一個位置的輸出過 `lm_head`,變成 248,320 個候選字的分數,取最高分就是下一個 token。

經典論文: [Attention Is All You Need](#) (Transformer 本體) · [An Image is Worth 16x16 Words](#) (ViT) · [SigLIP 2](#) (vision encoder) · [Qwen2-VL](#) (圖文怎麼併、MRoPE 怎麼標位置)

```
In [2]: model, processor = load("models/Qwen3.6-27B-4bit")
        tokenizer = processor.tokenizer

        llm = model.language_model
        inner = llm.model
        cfg = model.config.text_config
```

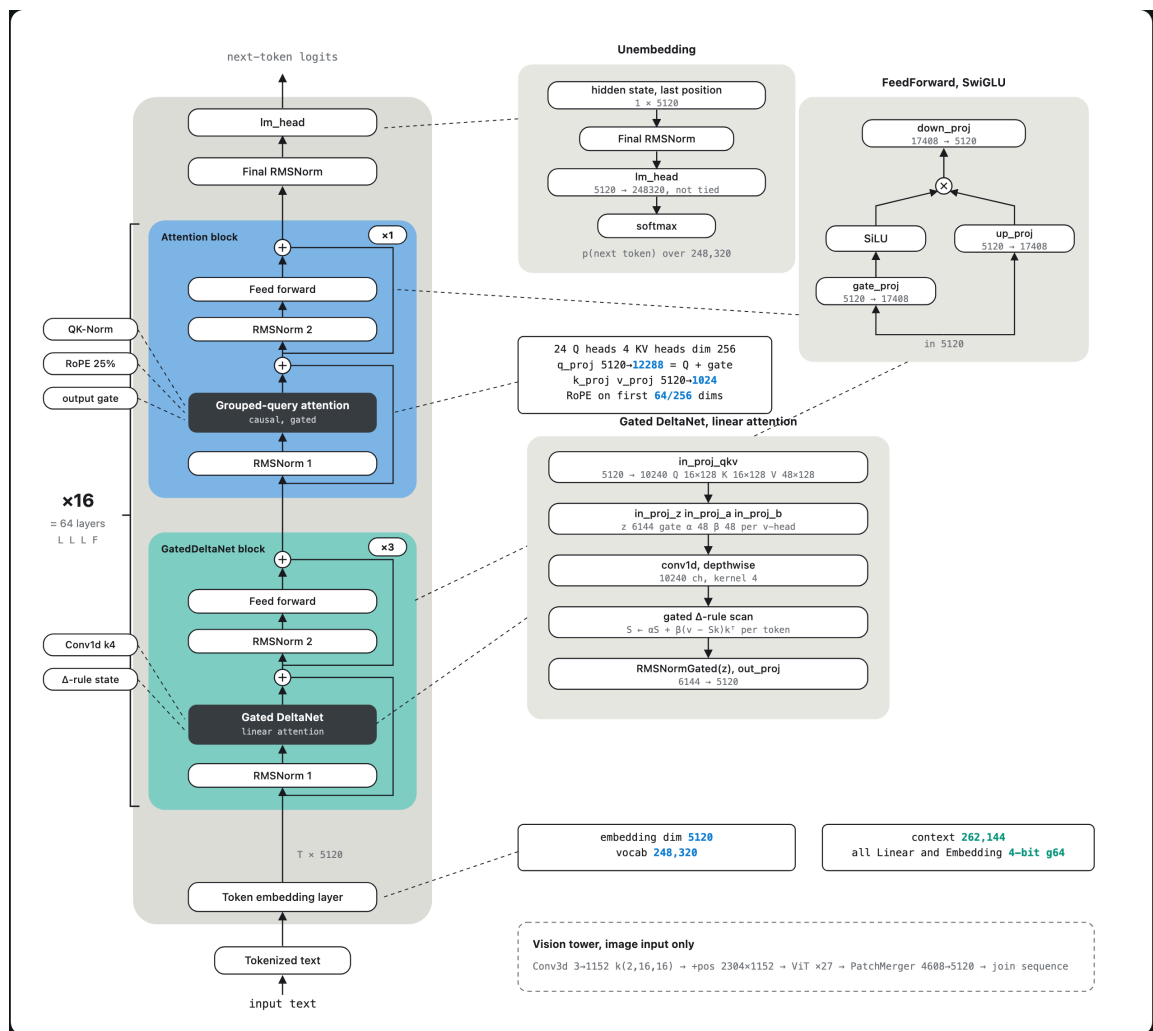
64 層由兩種 block 疊成,每 4 層一組。

```
In [3]: for i, layer in enumerate(inner.layers[:4]):
        kind = "GatedDeltaNet (linear attn)" if layer.is_linear else
        "Attention (full attn)"
        print(f"layer {i:2d}    {kind}")
        print(f"    ...      same pattern x 16  ->  {len(inner.layers)} layers
        total")
```

```
layer 0    GatedDeltaNet (linear attn)
layer 1    GatedDeltaNet (linear attn)
layer 2    GatedDeltaNet (linear attn)
layer 3    Attention (full attn)
```

... same pattern x 16 -> 64 layers total

兩種 decoder block



兩種 block 的外框一模一樣,都是在 residual stream 上加東西:

```
h <- h + mixer(RMSNorm(h))
h <- h + FFN(RMSNorm(h))
```

FFN 是 SwiGLU, 5120 升到 17408 再壓回 5120。差別只在中間那個 **mixer** 怎麼看前文。

Full attention block

就是標準 causal attention。Qwen 的設定: 24 個 Q head、4 個 KV head (GQA, 6 個 Q 共用一組 KV), head_dim 256。

三個細節: Q 和 K 各自先過 RMSNorm (QK-Norm) 穩住數值; RoPE 只轉前 64 維, 也就是 256 的 25%, 其餘維度不帶位置; 位置用 MRoPE 分三軸編, 圖片 patch 才有二維座標可標。

有一個地方跟課本版不同。q_proj 的輸出是 $24 \times 256 \times 2 = 12288$ 維, 一半當 Q, 另一半當 **output gate**:

```
attn = softmax(Q K^T / sqrt(256)) V
out = o_proj(attn * sigmoid(gate))
```

attention 算完之後先被 sigmoid gate 逐維縮放,才進 `o_proj` (6144 -> 5120) 寫回 residual stream。gate 讓這一層自己決定這次要寫多少進去。

GatedDeltaNet block

這個少見,講細一點。它不回頭看整串 sequence,而是維護一個固定大小的 **state**,邊讀邊更新。

每個 head 的 state 是一個 `128 x 128` 矩陣 `S` (value 維 x key 維)。可以想成一本查找表: `k` 是索引, `v` 是內容。

每讀一個 token 做四件事:

```
S <- g * S           # decay: 舊資訊按比例衰減
d <- beta * (v - S k) # delta rule: 新 value 減掉查表結果
S <- S + d k^T        # write: 只把差額寫回去
y <- S q             # read: 用 q 查表,得到這個位置的輸出
```

關鍵是中間兩步的 **delta rule**。它不硬把 `v` 塞進表裡,而是先用 `k` 查一次現在表裡有什麼 (`S k`),算出差額 `v - S k`,只補這個差額。所以同一個 key 被重複寫入時是覆蓋,不是疊加。

兩個 gate 控制力道,每個 head 各一個純量:

```
g = exp(-exp(A_log) * softplus(a + dt_bias)) # decay
gate, 落在 (0,1)
beta = sigmoid(b)                             # write
gate
```

`a`、`b` 都是從當前 token 算出來的 (`in_proj_a`、`in_proj_b`, 5120 -> 48),
`A_log`、`dt_bias` 是學來的參數。所以「忘多少、寫多強」是看內容決定,不是固定值。

進遞迴前還有一步: `q`、`k`、`v` 先過 depthwise causal conv1d (kernel 4) 加 SiLU,讓相鄰幾個 token 先混一下, `q` 和 `k` 再各自 normalize 並縮放。出來之後過一個 gated RMSNorm (gate 由 `in_proj_z` 給),最後 `out_proj` (6144 -> 5120) 寫回 residual stream。

Qwen 的設定: 16 個 `q/k` head、48 個 `v` head, `head_dim` 都是 128。所以一層的 state 是 `48 x 128 x 128`, 約 79 萬個數字,跟 **sequence 多長完全無關**。

經典論文: [RMSNorm](#) · [SwiGLU](#) · [RoPE](#) · [GQA](#) · [Linear Transformers](#) (linear attention 起點) · [Fast Weight Programmers](#) (delta rule 進到這一系) · [Gated Delta Networks](#) (這個 block 本身) · [Mamba-2](#) (同一系的 state space 做法)

差別在記憶成本: full attention 的 KV cache 隨 sequence 長大, GatedDeltaNet 的 state 不會。

```
In [4]: attn = inner.layers[3].self_attn
        gdn = inner.layers[0].linear_attn

        kv_per_token = 2 * attn.num_key_value_heads * attn.head_dim
        state_size = gdn.num_v_heads * gdn.head_v_dim * gdn.head_k_dim

        print(f"full attention    {attn.num_attention_heads} Q heads /
        {attn.num_key_value_heads} KV heads x {attn.head_dim}")
        print(f"                KV cache grows: {kv_per_token} numbers per token
        per layer")
        print(f"GatedDeltaNet    {gdn.num_v_heads} v heads x {gdn.head_v_dim} x
        {gdn.head_k_dim} state")
        print(f"                fixed: {state_size:,} numbers per layer, any
        sequence length")
        print(f"\nbreak-even at {state_size // kv_per_token} tokens; context limit
        is {cfg.max_position_embeddings:,}")
```

```
full attention    24 Q heads / 4 KV heads x 256
                  KV cache grows: 2048 numbers per token per layer
GatedDeltaNet    48 v heads x 128 x 128 state
                  fixed: 786,432 numbers per layer, any sequence
length
break-even at 384 tokens; context limit is 262,144
```

所以 3:1 的排法很直接:大部分層用固定大小的壓縮記憶,每 4 層插一次 full attention 回頭精確取用前文。

輸入長什麼樣

文字和影像各跑一次前處理,看實際的形狀。

```
In [5]: from PIL import Image

        text = "The capital of France is"
        text_ids = tokenizer.encode(text)

        print("TEXT")
        print(f"  input      : {text!r}")
        print(f"  token ids  : {text_ids}")
        print(f"  tokens     : {[tokenizer.decode([i]) for i in text_ids]}")

        img = Image.open("assets/qwen3_6_27b_arch_residual.png")
        vision = processor.image_processor(images=img, return_tensors="np")
        pixel_values, grid = vision["pixel_values"], vision["image_grid_thw"]
        t, h, w = grid[0]
        merge = processor.image_processor.merge_size

        print("\nIMAGE")
        print(f"  file       : {img.size[0]}x{img.size[1]} px")
        print(f"  grid t,h,w : {t},{h},{w} -> {t*h*w} patches")
        print(f"  pixel_values: {pixel_values.shape}    1536 = temporal 2 x RGB 3 x
        16 x 16")
        print(f"  after {merge}x{merge} merge: {(h//merge)*(w//merge)} vectors
        into the decoder")
```

TEXT

```
input      : 'The capital of France is'
token ids  : [760, 6511, 314, 9338, 369]
tokens     : ['The', ' capital', ' of', ' France', ' is']
```

IMAGE

```
file       : 1660x2260 px
grid t,h,w : 1,142,104 -> 14768 patches
pixel_values: (14768, 1536) 1536 = temporal 2 x RGB 3 x 16 x 16
after 2x2 merge: 3692 vectors into the decoder
```

英文那句是 5 個 token,一張圖攤開是 14768 個 patch,2x2 併完還剩 3692 個向量進 decoder。圖片吃 context 吃得很兇,這就是 merge 那一步存在的理由。

走一次 forward,確認模型正常。

```
In [6]: import mlx.core as mx

ids = mx.array(tokenizer.encode("The capital of France is"))[None]
logits = llm(ids).logits
next_id = int(mx.argmax(logits[0, -1]))

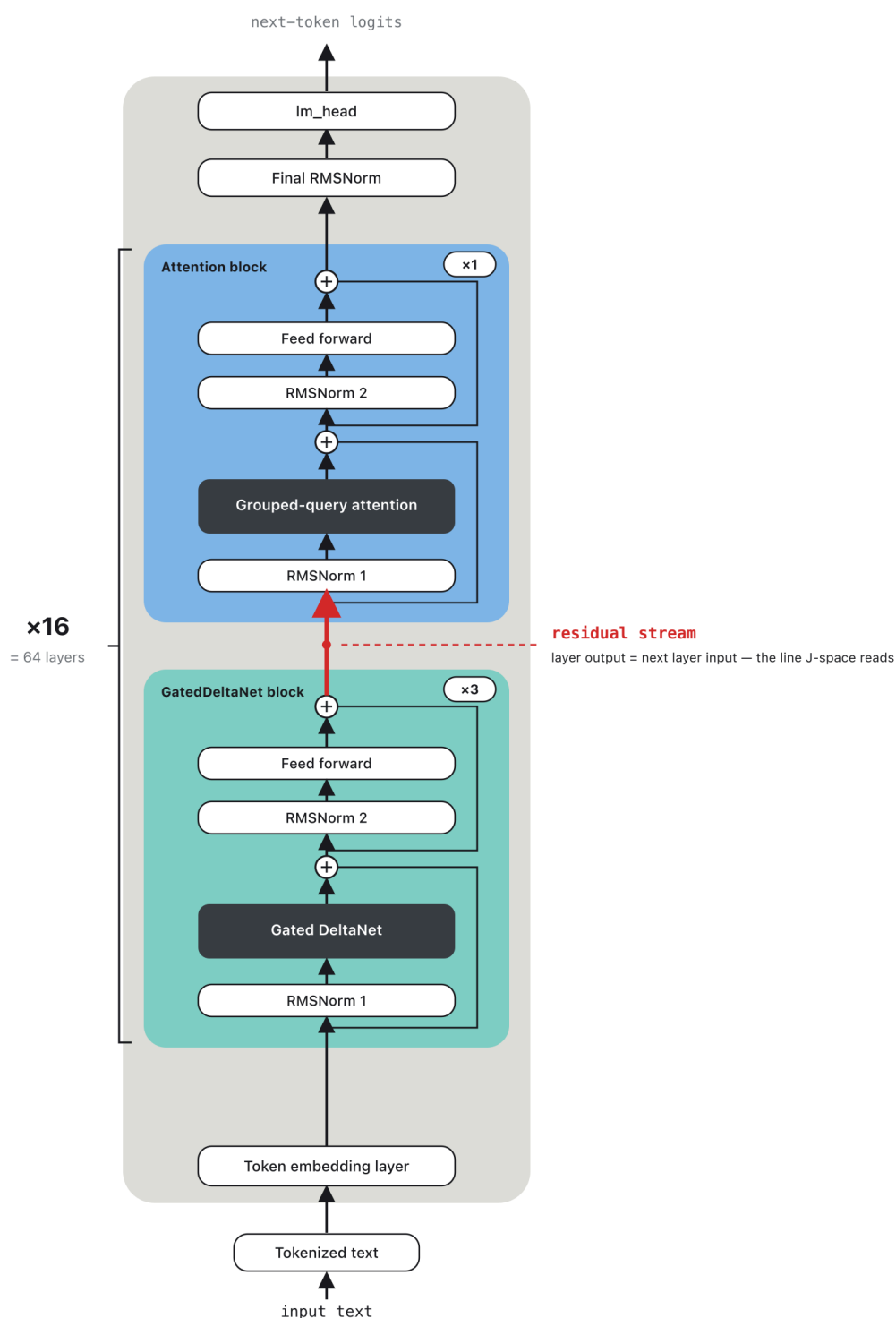
print(f"logits      : {logits.shape}")
print(f"next token  : {tokenizer.decode([next_id])!r}")

logits      : (1, 5, 248320)
next token  : ' Paris'
```


2. Logit lens

可解釋性 (interpretability) 想問的是:模型吐出答案的路上,中間到底發生了什麼。

Transformer 有個很好的切入點:**residual stream**。每一層不是重寫整條向量,而是把自己算的東西加回同一條向量上。因為每一層只是往上加東西,64 層的輸出都停在**同一個座標系**裡,前面寫進去的東西,後面每一層都讀得到 ([A Mathematical Framework for Transformer Circuits](#), Elhage et al. 2021)。



既然每層輸出都在同一個座標系上,那能不能直接拿尺去量?

這就是 **logit lens** 的想法 ([Interpreting GPT: the logit lens](#), nostalgebraist 2020)。模型最後那顆頭本來只讀最終的 h_L , 現在把它借來, 對**每一層**的 h_ℓ 都讀一次:

$$\text{LogitLens}(h_\ell) = W_U \cdot \text{norm}(h_\ell)$$

`norm` 是 final RMSNorm, W_U 是 `lm_head`。讀出來的就是「第 ℓ 層時, 模型以為下一個字是什麼」。

先把 residual 抽出來。這裡要手動一層層跑, 自己準備兩種 mask 和 MRoPE 位置, 這樣才跟官方 forward 完全一致。

```

In [7]: from mlx_vlm.models.qwen3_5.language import (
        _create_qwen3_5_attention_mask, _create_qwen3_5_ssm_mask)

dtype = inner.norm.weight.dtype
d_model = cfg.hidden_size
n_layers = len(inner.layers)

def encode(text):
    return mx.array(tokenizer.encode(text))[None]

def prep(h):
    fa = _create_qwen3_5_attention_mask(h, None)
    ssm = _create_qwen3_5_ssm_mask(h, None)
    pos = mx.tile(mx.arange(h.shape[1])[None, None, :], (3, 1, 1))
    return fa, ssm, pos

def set_linear_train(mode):
    """eval uses a non-differentiable metal kernel; train uses pure ops
    that mx.vjp
    can backprop through. Needed later for the Jacobian."""
    for layer in inner.layers:
        if layer.is_linear:
            layer.linear_attn.train(mode)

def residuals(ids):
    set_linear_train(False)
    h = inner.embed_tokens(ids)
    fa, ssm, pos = prep(h)
    out = []
    for layer in inner.layers:
        h = layer(h, mask=(ssm if layer.is_linear else fa),
                  cache=None, position_ids=pos, position_embeddings=None)
        out.append(h)
    return out

def readout(h):
    return llm.lm_head(inner.norm(h))

res = residuals(encode("The capital of France is"))
print(f"{len(res)} residuals, each {tuple(res[0].shape)} = (batch, tokens,
d_model)")

```

64 residuals, each (1, 5, 5120) = (batch, tokens, d_model)

```
In [8]: prompt = "The capital of France is"
ids = encode(prompt)
res = residuals(ids)
last = ids.shape[1] - 1

answer = int(mx.argmax(readout(res[-1])[0, last]))
print(f"prompt = {prompt!r}    answer = {tokenizer.decode([answer])!r}\n")

print("layer    top-1                p(top1)    p(answer)")
for l in range(0, n_layers, 4):
    logits = readout(res[l])[0, last].astype(mx.float32)
    probs = mx.softmax(logits)
    top = int(mx.argmax(logits))
    print(f"L{l:2d}    {tokenizer.decode([top])!r:18s}
{float(probs[top]):7.3f}    {float(probs[answer]):7.3f}")

prompt = 'The capital of France is'    answer = ' Paris'
```

layer	top-1	p(top1)	p(answer)
L 0	' '	0.010	0.000
L 4	' '	0.013	0.000
L 8	'...'	0.008	0.000
L12	'...'	0.005	0.000
L16	'...'	0.008	0.000
L20	'陷'	0.006	0.000
L24	'...'	0.006	0.000
L28	'_____'	0.008	0.000
L32	'zl'	0.007	0.000
L36	'_____'	0.032	0.000
L40	'...'	0.029	0.000
L44	'...'	0.070	0.000
L48	'_____'	0.012	0.000
L52	'_____'	0.023	0.000
L56	' Paris'	0.225	0.225
L60	' Paris'	0.858	0.858

讀出來的字很晚才有意義。前 50 幾層的 top-1 是 ' '、'...'、'_____'、'陷'、'zl' 這類碎片, p(answer) 一直是 0.000,到 L56 才冒出 ' Paris' 的 0.225,L60 是 0.858。

但這是 **logit lens** 的問題,不一定是模型的實情。把 h_ℓ 直接丟進最後那顆頭, 等於偷偷假設「從第 ℓ 層到終點的那幾十層什麼都不做」:

$$\text{LogitLens}(h_\ell) = W_U \cdot \text{norm}(h_\ell) \quad \Longleftrightarrow \quad \frac{\partial h_L}{\partial h_\ell} \approx I$$

對最後幾層還堪用,對中層就太粗了。答案可能早就算好,只是還沒轉到最後那顆頭讀得到的方向上。

3. Jacobian lens

Anthropic 的做法很直接:那個被隨手當成 I 的東西,把它真的算出來。

$$J_\ell = \frac{\partial h_L}{\partial h_\ell} \quad \text{JLens}(h_\ell) = W_U \cdot \text{norm}(J_\ell h_\ell)$$

J_ℓ 是「第 ℓ 層的 residual 動一點,最終的 residual 會怎麼跟著動」的一階近似,一個 5120×5120 的矩陣。它在某個輸入、某個位置附近展開,不是一個全域的座標轉換。乘上它,等於把 h_ℓ 搬到最終 residual 附近再用同一顆頭去讀,跟 logit lens 的差別 只有這一步。

官方實作:anthropics/jacobian-lens。社群解說:[J-Space: Yet Another LLM Mind Reader?](#)。一份獨立重現與批評:[A Review of Anthropic's Global Workspace Paper](#)。

從零算 J_ℓ

要拿到 J_ℓ ,用 `mx.vjp` 反傳最直接:在第 `src` 層之後注入一個 delta, 看第 `tgt` 層的 residual 怎麼變。

有兩個實務細節:

1. GatedDeltaNet 在 eval 模式走的是不可微的 metal kernel,所以算之前要切到 train 模式 (純 ops 的 chunked scan),算完切回來。
2. chunked scan 內部會呼叫 `mx.async_eval`,那在 `mx.vjp` 裡不合法,暫時換成 `no-op`。

矩陣有 5120 列,全算太慢,這裡只算前 64 列證明它是真的算得出來。

```

In [9]: import contextlib
import time

SKIP_FIRST = 16  # early positions are BOS and formatting, readout there
is noise
TARGET = n_layers - 1

@contextlib.contextmanager
def no_async_eval():
    orig = mx.async_eval
    mx.async_eval = lambda *a, **k: None
    try:
        yield
    finally:
        mx.async_eval = orig

def forward_with_injection(ids, delta, src, tgt):
    h = inner.embed_tokens(ids)
    fa, ssm, pos = prep(h)
    for l, layer in enumerate(inner.layers):
        h = layer(h, mask=(ssm if layer.is_linear else fa),
                    cache=None, position_ids=pos, position_embeddings=None)
        if l == src:
            h = h + delta
        if l == tgt:
            return h
    return h

def jacobian_rows(ids, src, tgt, n_rows):
    seq = ids.shape[1]
    valid = mx.array(list(range(SKIP_FIRST, seq - 1)), dtype=mx.int32)
    delta = mx.zeros((n_rows, seq, d_model), dtype=dtype)
    onehot = (mx.arange(d_model)[None, :] == mx.arange(n_rows)[:,
None]).astype(mx.float32)
    posmask = (mx.arange(seq)[None, :] == valid[:, None]).any(axis=0)
    cot = (onehot[:, None, :] * posmask[None, :,
None]).astype(mx.float32).astype(dtype)

    set_linear_train(True)
    with no_async_eval():
        (_,), (g,) = mx.vjp(
            lambda d: forward_with_injection(ids, d, src, tgt), (delta,),
(cot,))
        rows = mx.take(g, valid, axis=1).mean(axis=1).astype(mx.float32)
        mx.eval(rows)
    set_linear_train(False)
    return rows

long_prompt = ("The history of the Roman Empire spans several centuries and
includes "
               "many emperors, wars, and cultural achievements that shaped
the ancient world.")
LAYER = 54

t0 = time.perf_counter()

```

```
rows = jacobian_rows(mx.array(tokenizer.encode(long_prompt)[:48])[None],
                      src=LAYER, tgt=TARGET, n_rows=64)
print(f"J[L{LAYER}] first 64 rows: {tuple(rows.shape)} in {time.perf_counter()-t0:.0f}s")
print(f"diagonal entry J[0,0] = {float(rows[0, 0]):.3f}    row norm = {float(mx.linalg.norm(rows[0])):.3f}")
```

J[L54] first 64 rows: (64, 5120) in 18s
 diagonal entry J[0,0] = 0.949 row norm = 1.136

J[0,0] = 0.949, 不是 1。單看這一個對角元素, logit lens 把 J_ℓ 當成 I 不算離譜。差的那一點會不會改變讀出來的字, 是下一段的事。

換成公開的 lens

上面是用一條 prompt 算的。Anthropic 的做法是對很多條 prompt 取平均, 得到一個不綁特定輸入的 lens。這裡直接載 Neuronpedia 放出來的擬合結果 ([neuronpedia/jacobian-lens](https://neuronpedia.com/jacobian-lens)), 比對從零算的那 64 列跟它像不像。

```
In [10]: _lens = mx.load("models/Qwen3.6-27B-4bit/jlens.npz")
src_layers = [int(x) for x in _lens["__source_layers__"].tolist()]
n_prompts = int(_lens["__n_prompts__"].tolist()[0])
Jlens = {l: _lens[f"J_{l}"] for l in src_layers}

print(f"public lens: layers {src_layers[0]}..{src_layers[-1]} ({len(src_layers)}), "
      f"fitted on {n_prompts} prompts")

a = rows.reshape(-1)
b = Jlens[LAYER].astype(mx.float32)[:64].reshape(-1)
cos = float((a @ b) / (mx.linalg.norm(a) * mx.linalg.norm(b)))
print(f"cos(our 1-prompt rows, public {n_prompts}-prompt lens) = {cos:.3f}")

def transport(h, l):
    """Move a layer-l residual into the final-residual basis: J_l @ h."""
    J = Jlens[l]
    return (h.astype(J.dtype) @ J.T).astype(dtype)
```

public lens: layers 0..62 (63), fitted on 1000 prompts
 cos(our 1-prompt rows, public 1000-prompt lens) = 0.894

cosine 0.894。比的只是 L54 的前 64 列, 一個數字說不上 J_ℓ 與輸入無關, 但一條 prompt 算出來的方向已經跟一千條擬合的結果大致同向。

兩把尺並排

同一次 forward, 同一顆頭, 唯一差別是有沒有乘 J_ℓ 。

```
In [11]: prompt = "The capital of France is"
ids = encode(prompt)
res = residuals(ids)
last = ids.shape[1] - 1
answer = int(mx.argmax(readout(res[-1])[0, last]))

print(f"{prompt!r} -> {tokenizer.decode([answer])!r}\n")
print("layer    logit p(ans)    jacobian p(ans)    jacobian top-1")
for l in range(48, n_layers):
    if l not in Jlens:
        continue
    h = res[l][0, last]
    lp = float(mx.softmax(readout(h).astype(mx.float32))[answer])
    jlogits = readout(transport(h, l)).astype(mx.float32)
    jp = float(mx.softmax(jlogits)[answer])
    mark = "    <--" if jp > 0.5 > lp else ""
    print(f"L{l:2d}    {lp:.3f}    {jp:.3f}    {jp:.3f}    "
          f"{tokenizer.decode([int(mx.argmax(jlogits))])!r}{mark}")
```

'The capital of France is' -> ' Paris'

layer	logit p(ans)	jacobian p(ans)	jacobian top-1
L48	0.000	0.000	' _____ '
L49	0.000	0.004	' _____ '
L50	0.000	0.001	' _____ '
L51	0.000	0.001	' _____ '
L52	0.000	0.001	' _____ '
L53	0.000	0.012	' _____ '
L54	0.001	0.086	' _____ '
L55	0.088	0.895	' Paris' <--
L56	0.237	0.961	' Paris' <--
L57	0.520	0.972	' Paris'
L58	0.922	0.990	' Paris'
L59	0.856	0.987	' Paris'
L60	0.858	0.989	' Paris'
L61	0.940	0.980	' Paris'
L62	0.247	0.848	' Paris' <--

關鍵那一行是 **L55**:同一條 residual、同一顆讀出頭,logit lens 給 0.09, Jacobian lens 給 0.90。top-1 也差一層,Jacobian lens 在 L55 就讀到 ' Paris'。

先做一個對照,不然這個結果不算數

公開 lens 在擬合時會跳過每條 prompt 的前 16 個 position (issue #5 就在講這件事)。而上面那條 prompt 只有 5 個 token,我們讀的是 position 4,剛好落在 lens 沒擬合過的區間。

所以要補一個對照:在前面墊一段無關的話,把要讀的位置推到 16 以後,看結果還在不在。


```
In [12]: FILLER = ("Here are some notes about geography that provide background
context "
            "for the question that follows this introductory sentence. ")

def compare_lenses(prompt, layers=(50, 53, 54, 55, 56, 58)):
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    ans = int(mx.argmax(readout(res[-1])[0, last].astype(mx.float32)))
    print(f"read position {last:>2}    answer {tokenizer.decode([ans])!r}")
    for l in layers:
        if l not in Jlens:
            continue
        h = res[l][0, last]
        lp = float(mx.softmax(readout(h).astype(mx.float32))[ans])
        jp = float(mx.softmax(readout(transport(h, l)).astype(mx.float32))
[ans])
        print(f"  L{l:2d}    logit {lp:.3f}    jacobian {jp:.3f}")

print("SHORT prompt, read position is inside the unfitted range")
compare_lenses("The capital of France is")
print("\nPADDED prompt, read position is inside the fitted range")
compare_lenses(FILLER + "The capital of France is")
```

SHORT prompt, read position is inside the unfitted range

read position 4 answer ' Paris'

L50	logit 0.000	jacobian 0.001
L53	logit 0.000	jacobian 0.012
L54	logit 0.001	jacobian 0.086
L55	logit 0.088	jacobian 0.895
L56	logit 0.237	jacobian 0.961
L58	logit 0.922	jacobian 0.990

PADDED prompt, read position is inside the fitted range

read position 23 answer ' Paris'

L50	logit 0.000	jacobian 0.217
L53	logit 0.001	jacobian 0.765
L54	logit 0.002	jacobian 0.796
L55	logit 0.117	jacobian 0.982
L56	logit 0.321	jacobian 0.979
L58	logit 0.970	jacobian 0.998

落差沒消失,反而更大。擬合過的位置上,L53 是 logit 0.001 對 Jacobian 0.765, L50 是 0.000 對 0.217。

兩張表合起來才下得了結論:這條 prompt 的答案 L50 就已經在 residual 裡, logit lens 要到 L58 才讀到 0.97。短 prompt 那版是低估落差,不是製造落差。

4. 一個有趣的例子

用中文問同一件事。

```
In [13]: PARIS = {11751, 57590, 109705} # 'Paris', 'Paris', and the Chinese token

def lens_trace(prompt, layers, k=3):
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    print(f"{prompt!r}    output top-1 = "
          f"{tokenizer.decode([int(mx.argmax(readout(res[-1]))[0,
last]))])!r}")
    print("                jacobian top-3                logit top-3")
    for l in layers:
        if l not in Jlens:
            continue
        h = res[l][0, last]
        j = mx.argsort(-readout(transport(h, l)).astype(mx.float32))[:k]
        g = mx.argsort(-readout(h).astype(mx.float32))[:k]
        js = " ".join(f"{tokenizer.decode([int(t)])!r}" for t in j)
        gs = " ".join(f"{tokenizer.decode([int(t)])!r}" for t in g)
        print(f"  L{l:2d}  {js:40s}  {gs}")

lens_trace("法国的首都是", range(40, 62, 4))
```

'法国的首都是' output top-1 = '哪个'

	jacobian top-3	logit top-3
L40	'Paris' 'city' 'cities'	'...' '____' '____'
L44	'____' 'city' 'cities'	'...' '____' '<u'
L48	'____' '____' '____'	'<u' '____' '____'
L52	'?' '____' '____'	'gì' '什么' '哪个'
L56	'哪里' '哪个' 'Paris'	'哪里' '哪个' '否'
L60	'Paris' '巴黎' 'Paris'	'哪个' '哪里' '巴黎'

中文問句, Jacobian lens 在 **L40** 就把英文的 'Paris' 排第一, 同一層的 logit lens 還是 '...' 這種碎片。中文的 '巴黎' 要到 L60 才進 top-3, 而模型表面上要說的下一個字是 '哪个', 在把句子接完(「法国的首都是哪个城市」)。

這條 prompt 的順序是: 答案先以英文浮出來, 中文晚得多, 表面還在造句。方向跟 Anthropic 在 [On the Biology of a Large Language Model](#) 講的多語言迴路一致, 但一條 prompt 證不了「不綁語言的概念空間」, 只能說看到的順序不衝突。

換幾個語言, 看 Paris 最早在哪一層進 top-5。

```
In [14]: def first_hit(prompt, targets, topk=5):
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    fj = fl = None
    for l in range(n_layers):
        if l not in Jlens:
            continue
        h = res[l][0, last]
        j = set(int(t) for t in mx.argsort(-readout(transport(h,
l)).astype(mx.float32))[:topk].tolist()))
        g = set(int(t) for t in mx.argsort(-readout(h).astype(mx.float32))
[:topk].tolist()))
        if fj is None and j & targets:
            fj = l
        if fl is None and g & targets:
            fl = l
    return fj, fl

for p in ["The capital of France is", "法国的首都是",
        "La capitale de la France est", "フランスの首都は"]:
    fj, fl = first_hit(p, PARIS)
    gap = "" if None in (fj, fl) else f" {fl - fj} layers earlier"
    print(f"{p!r:34s} jacobian L{fj} logit L{fl}{gap}")

'The capital of France is' jacobian L54 logit L55 1
layers earlier
'法国的首都是' jacobian L23 logit L58 35
layers earlier
'La capitale de la France est' jacobian L21 logit L57 36
layers earlier
'フランスの首都は' jacobian L39 logit L56
17 layers earlier
```

三條非英文 prompt 都是這樣,中文和法文最誇張:Jacobian lens 在 L21 到 L23 就把 Paris 排進前五,logit lens 要等到 L57 以後。

英文問句用這個指標看不出差距(L54 對 L55),因為英文的 Paris 本來就不會晚進 top-5; 要看機率才清楚,前面那張表的 L55 是 0.09 對 0.90。

測的是三個語言各一條 prompt,推不到「所有非英文問句」。

5. J-space 到底是什麼

到這裡讀出的都是「哪個 token 分數最高」。論文的 **J-space** 是更強的一個宣稱。

先定義一個東西:詞表裡每個 token t 在第 ℓ 層的 residual 裡對應一個方向

$$v_t = (W_U J_\ell)_t$$

也就是 $W_U J_\ell$ 的第 t 列,讀出時那個 token 的分數就是內積 $\langle v_t, h \rangle$ 。前面每一次 readout,其實都是在算所有 token 的這個內積。

J-space 的宣稱是:residual 的一小塊可以寫成少數幾個這種方向的**非負**組合,

$$h_\ell \approx \sum_{t \in S} a_t v_t, \quad a_t \geq 0, \quad |S| \leq 25$$

S 就是「此刻在 workspace 裡的東西」。非負是因為這些方向代表「有這個概念」,負係數沒有對應的意思。

實作上詞表有 248320 個方向,不可能全試。做法是先用 readout 分數挑前 k 個候選,再解一個非負最小平方(投影梯度,步長取 $1/\|G\|_F$ 保證不會發散)。

兩個對照: k 掃 5 / 25 / 100,看解釋量會不會一路長上去;以及隨機挑 25 個 token 方向當底線。

```
In [15]: lh = llm.lm_head
```

```
def unembed_row(token):
    """One row of W_U, dequantized out of the 4-bit lm_head."""
    i = mx.array([token])
    return mx.dequantize(lh.weight[i], lh.scales[i], lh.biases[i],
                        group_size=lh.group_size,
                        bits=lh.bits)[0].astype(mx.float32)

def jlens_vec(token, l):
    """The J-lens direction of one token: row t of W_U @ J_l."""
    return unembed_row(token) @ Jlens[l].astype(mx.float32)

def nnls(V, h, iters=4000):
    """min ||h - V a|| with a >= 0, by projected gradient.

    The step is 1 / ||G||_F, and the Frobenius norm bounds the spectral
    norm from
    above, so the iteration cannot diverge (a bad step size showed up as
    negative
    explained variance the first time round).
    """
    G = V.T @ V
    b = V.T @ h
    L = float(mx.sqrt(mx.sum(G * G)))
    a = mx.zeros((V.shape[1],))
    for _ in range(iters):
        a = mx.maximum(a - (G @ a - b) / L, 0.0)
    return a

def jspace_fit(h, l, k=25, tokens=None):
    """Fit h with k J-lens directions: the k best readout scores unless
    given."""
    if tokens is None:
        sc = readout(transport(h[None, None], l))[0, 0].astype(mx.float32)
        tokens = [int(t) for t in mx.argsort(-sc)[:k].tolist()]
    V = mx.stack([jlens_vec(t, l) for t in tokens], axis=1)
    a = nnls(V, h)
    r = h - V @ a
    return 1.0 - float(mx.sum(r * r) / mx.sum(h * h)), tokens, a

ids = encode("The capital of France is")
res = residuals(ids)
last = ids.shape[1] - 1
RANDOM_25 = [int(x) for x in mx.random.randint(0, 200000, (25,)),

key=mx.random.key(0)).tolist()]

for l in (48, 55, 60):
    h = res[l][0, last].astype(mx.float32)
    cols, note = [], ""
    for k in (5, 25, 100):
        ev, toks, a = jspace_fit(h, l, k)
        cols.append(f"k={k}: {ev * 100:5.1f}%")
```

```

        if k == 25:
            note = (f"nonzero {int(mx.sum(a > 1e-6))}/25 "
                    + " ".join(repr(tokenizer.decode([t])) for t in
                                toks[:6]))
            ev_r, _, _ = jspace_fit(h, l, tokens=RANDOM_25)
            print(f"L{l} " + " ".join(cols) + f"    random-25: {ev_r *
100:4.1f}%")
            print(f"        {note}")

```

```

L48  k=5:   1.2%   k=25:   1.5%   k=100:   3.6%   random-25:  0.2%
      nonzero 7/25   '____' ' ' '____' ' ' '____' ' ' '____' ' ' '____' '
L55  k=5:   5.3%   k=25:   6.7%   k=100:   9.6%   random-25:  0.1%
      nonzero 12/25  ' Paris' 'Paris' '____' ' ' '____' ' ' '____' '
L60  k=5:   5.8%   k=25:   8.1%   k=100:  10.7%   random-25:  0.0%
      nonzero 12/25  ' Paris' 'Paris' ' London' ' Berlin' ' paris'
      '巴黎'

```

三件事。

量很小。 25 個方向在 L48 只解釋 1.5%, L55 是 6.7%, L60 是 8.1%。就算放到 100 個方向, 最多也才 10.7%。residual 的絕大部分不在這個由 J-lens 方向撐出來的東西裡面。論文自己說的也是這個量級, 所以這不是實作沒做好, 是這個物件本來就只佔一小塊。

但不是隨便挑都行。 隨機 25 個 token 方向只解釋 0.0% 到 0.2%, 差了一到兩個數量級。挑出來的那 25 個是有結構的。

它比 25 還稀疏。 非負最小平方自己把一半的候選壓到 0: L55 和 L60 都只有 12 個係數 是正的, L48 只有 7 個。 $k = 25$ 是上限, 不是實際用到的數目。

看 L60 挑到的 token 也有意思: ' Paris'、'Paris'、' London'、' Berlin'、' paris'、'巴黎'。同一個概念的不同拼法, 加上同類的其他首都。

順帶把「J-space」這個名字的問題講清楚: 非負又有稀疏上限, 這不是一個線性子空間, 不能講「投影到 J-space」。照論文自己的 methods, 它是一堆多面錐的聯集。

6. 換掉 J-space 裡的概念

前面都在讀,這一節改。

對齊原文的實驗:四個 prompt 分別問 France 的首都、語言、所屬洲、貨幣,在 J-space 裡把 France 換成 China,四個情境用完全相同的介入。四個答案一起改的話,最簡單的解釋是它們讀的是同一份表徵。

做法是論文的 lens coordinate patching,用的就是 §5 那些 v_t 。換概念是三步:

$$V = [v_s \ v_t], \quad c = V^\dagger h, \quad h \leftarrow h + \alpha V (\sigma(c) - c)$$

c 是 h 在這兩個方向上的座標, σ 把兩個座標交換。跟 $\text{span}\{v_s, v_t\}$ 正交的成分完全不動,所以幾何上只動了這個平面;語義上是不是只有 France 這一個概念被換掉,要看輸出,公式本身不保證。

$V^\dagger$ 是 pseudo-inverse。不能直接拿內積當座標:J-lens 向量 overcomplete 而且彼此不正交,下面會看到 v_{France} 跟 v_{China} 的 cosine 有 0.44, 直接取內積會把重疊的部分算兩次。

```
In [16]: def coords(x, vs, vt):
          V = mx.stack([vs, vt], axis=1)
          return V, mx.linalg.pinv(V, stream=mx.cpu) @ x

def swap(x, vs, vt, alpha):
    V, c = coords(x, vs, vt)
    return x + alpha * (V @ (c[:-1] - c))

def add_only(x, vs, vt, alpha):
    V, c = coords(x, vs, vt)
    return x + alpha * (c[0] - c[1]) * vt

FRANCE, CHINA = tokenizer.encode(" France")[0], tokenizer.encode(" China")
[0]
vf, vc = jlens_vec(FRANCE, 48), jlens_vec(CHINA, 48)
cos = float(vf @ vc / (mx.linalg.norm(vf) * mx.linalg.norm(vc)))
print(f"v_France at L48: {tuple(vf.shape)}, norm
      {float(mx.linalg.norm(vf)).1f}")
print(f"cos(v_France, v_China) = {cos:+.3f}")
```

v_France at L48: (5120,), norm 1.4

cos(v_France, v_China) = +0.441

三個選擇決定成不成:

- 層:L40 到 L55。這是掃出來的:更早的層改了幾乎沒反應,更晚的層改了會壞事(見下)。
- 位置:只改 France 那個 token,不碰最後一個位置。最後幾層的 J-lens 方向是「準備要說出口的字」,在那裡動手 model 會直接把 China 說出來,而不是拿它去推理。
- 強度: $\alpha = 1$ 是嚴格的交換, $\alpha = 2$ 是推過頭的版本,兩個都跑。

另外跑一個對照組 add_only :只加上 China 的成分,不把 France 拿掉。

```
In [17]: BAND = [l for l in Jlens if 40 <= l < 56]
ALPHA = 2.0
```

```
def patched(ids, pos, alpha, edit=swap):
    h = inner.embed_tokens(ids)
    fa, ssm, p = prep(h)
    for l, layer in enumerate(inner.layers):
        h = layer(h, mask=(ssm if layer.is_linear else fa),
                    cache=None, position_ids=p, position_embeddings=None)
    if alpha and l in BAND:
        h[0, pos] = edit(h[0, pos].astype(mx.float32),
                        jlens_vec(FRANCE, l), jlens_vec(CHINA, l),
                        alpha).astype(dtype)
    return readout(h)[0, ids.shape[1] - 1].astype(mx.float32)

def p_of(sc, s):
    return float(mx.softmax(sc)[tokenizer.encode(s)[0]])

def top1(sc):
    return tokenizer.decode([int(mx.argmax(sc))])

TASKS = [("The capital of France is", " Paris", " Beijing"),
          ("The language spoken in France is", " French", " Chinese"),
          ("The continent that contains France is", " Europe", " Asia"),
          ("The currency of France is called the", " Euro", " Yuan")]

for prompt, orig, new in TASKS:
    ids = encode(prompt)
    toks = [tokenizer.decode([int(t)]) for t in ids[0].tolist()]
    pos = next(i for i, t in enumerate(toks) if "France" in t)
    print(f"{prompt!r}")
    for tag, sc in [("before", patched(ids, pos, 0)),
                    ("swap a=1", patched(ids, pos, 1.0)),
                    ("swap a=2", patched(ids, pos, ALPHA)),
                    ("add only a=2", patched(ids, pos, ALPHA, add_only))]:
        print(f"    {tag}    {top1(sc)!r:11s} p({orig})={p_of(sc, orig):.3f}"
              f"    p({new})={p_of(sc, new):.3f}")
```

'The capital of France is'

before	' Paris'	p(Paris)=0.617	p(Beijing)=0.000
swap a=1	' Paris'	p(Paris)=0.497	p(Beijing)=0.034
swap a=2	' Beijing'	p(Paris)=0.006	p(Beijing)=0.383
add only a=2	' Paris'	p(Paris)=0.427	p(Beijing)=0.066

'The language spoken in France is'

before	' French'	p(French)=0.539	p(Chinese)=0.000
swap a=1	' French'	p(French)=0.528	p(Chinese)=0.001
swap a=2	' Chinese'	p(French)=0.004	p(Chinese)=0.222
add only a=2	' French'	p(French)=0.461	p(Chinese)=0.002

'The continent that contains France is'

before	' Europe'	p(Europe)=0.153	p(Asia)=0.001
swap a=1	' Europe'	p(Europe)=0.135	p(Asia)=0.007


```

swap a=2      ' Asia'      p( Europe)=0.090  p( Asia)=0.096
add only a=2  ' Europe'    p( Europe)=0.138  p( Asia)=0.005
'The currency of France is called the'
before       ' euro'       p( Euro)=0.111   p( Yuan)=0.000
swap a=1     ' euro'       p( Euro)=0.101   p( Yuan)=0.017
swap a=2     ' yuan'       p( Euro)=0.001   p( Yuan)=0.060
add only a=2 ' euro'       p( Euro)=0.097   p( Yuan)=0.036

```

四個問題,同一個介入,答案全部跟著改:Paris 變 Beijing、French 變 Chinese、Europe 變 Asia、euro 變 yuan。四個都要 $\alpha = 2$ 。

$\alpha = 1$ 是嚴格的交換,方向對但推不過去: `p(Beijing)` 從 0.000 到 0.034, top-1 還是 'Paris', 四個任務一個都沒翻。

`add only a=2` 是對照組,只加上 China 的成分、不把 France 拿掉,四個也都沒翻, `p(Beijing)` 只有 0.066,交換版是 0.383。所以「把 France 拿掉」這一半是必要的。

幅度要留意。Neel Nanda 團隊在**同一顆 model 上複現**時也只拿到微弱但為正的因果效果,這是一顆 27B 的 4-bit 量化 model,不是 Sonnet 4.5。Asia 那一列最勉強,0.096 對 Europe 的 0.090,剛好過去而已。

7. 論文的五個 workspace 面向

論文不是只說「中間層有東西」。它給了五個功能面向,一個一個測,才算得上 workspace (原文圖 2):

面向	問題
verbal report	問它在想什麼,它說得出來嗎
directed modulation	叫它想 X、或叫它不要想 X, J-space 跟著動嗎
internal reasoning	它會拿 J-space 裡的東西去推理,還是只是存著
flexible generalization	同一個概念,換到不同任務還通嗎
selectivity	只有進 workspace 的內容才影響行為嗎

flexible generalization 就是 §6:一個介入,四個任務一起改。這一節把剩下四個 放到 Qwen3.6-27B-4bit 上跑,順便加一個論文用來驗因果的手法:把概念**消除**。

先把介入寫成一個通用的 forward。`edits` 是一串 (position, 函式),每一層都套上去。

```

In [18]: SCAN = [l for l in JlenS if 40 <= l < 62 and l % 2 == 0]

def tid(word):
    """First token id of a spelling."""
    return tokenizer.encode(word)[0]

def edited(ids, edits=(), layers=BAND):
    """Forward pass with edits = [(position, fn)] applied at every layer in
    `layers`."""
    h = inner.embed_tokens(ids)
    fa, ssm, p = prep(h)
    for l, layer in enumerate(inner.layers):
        h = layer(h, mask=(ssm if layer.is_linear else fa),
                  cache=None, position_ids=p, position_embeddings=None)
        if l in layers:
            for pos, fn in edits:
                h[0, pos] = fn(h[0, pos].astype(mx.float32),
                               l).astype(dtype)
    return readout(h)[0, ids.shape[1] - 1].astype(mx.float32)

def best_readout(ids, words, layers=SCAN):
    """Largest J-lens probability of any spelling, over every layer and
    position."""
    res = residuals(ids)
    best = (0.0, None, None)
    for l in layers:
        sm = mx.softmax(readout(transport(res[l], l)).astype(mx.float32),
                        axis=-1)[0]
        for w in words:
            col = sm[:, tid(w)]
            p = float(mx.max(col))
            if p > best[0]:
                best = (p, l, int(mx.argmax(col)))
    return best

def probs(sc, *pairs):
    sm = mx.softmax(sc)
    return " ".join(f"p({n})={max(float(sm[tid(w)]) for w in ws):.3f}"
                    for n, ws in pairs)

def where(ids, word):
    toks = [tokenizer.decode([int(t)]) for t in ids[0].tolist()]
    return next(i for i, t in enumerate(toks) if word in t)

print(f"scan layers: {SCAN[0]}..{SCAN[-1]} ({len(SCAN)})    patch band: "
      f"L{BAND[0]}-L{BAND[-1]}")

```

scan layers: 40..60 (11) patch band: L40-55

7.1 定向調節:叫它想,叫它不要想

同一句抄寫任務,後面接不同的指令,量 J-space 裡有沒有那個概念。

兩組:

- **白熊**。 `bear` 在兩個 prompt 裡都出現,所以「因為輸入有這個字才讀到」的效應會互相抵銷,差的是指令。
- **lemon**。指令說的是 citrus fruit, `lemon` 從頭到尾沒出現在 prompt 裡,讀到它就不能用 input-copying 解釋。對照組把指令換成想呼吸,測「任何指令都會抬高一切」。

讀法是掃 L40 到 L60 每一層、每一個位置,取最大的機率。

```
In [19]: S = 'Write the sentence "The old painting hung crookedly on the wall."'

CASES = [
    ("bear (in the prompt)", ("bear", "bears"), [
        ("not mentioned", S),
        ("think about it", S + " While you write, think about a white bear."),
        ("do NOT think of it", S + " While you write, do not think about a white bear.")]],
    ("lemon (never in the prompt)", ("lemon", "lemons"), [
        ("no instruction", S),
        ("think about citrus", S + " While you write, think about citrus fruit."),
        ("think of breathing", S + " While you write, think about your breathing.")]],
]

for target, words, cases in CASES:
    print(f"target {target}")
    for tag, prompt in cases:
        p, l, pos = best_readout(encode(prompt), words)
        spot = f"L{l} pos{pos}" if l is not None else "-"
        print(f"    {tag}    max p = {p:.4f}    at {spot}")
```

target bear (in the prompt)

not mentioned	max p = 0.0001	at L60 pos5
think about it	max p = 0.0075	at L54 pos23
do NOT think of it	max p = 0.1079	at L60 pos24

target lemon (never in the prompt)

no instruction	max p = 0.0001	at L48 pos3
think about citrus	max p = 0.0114	at L58 pos22
think of breathing	max p = 0.0001	at L48 pos3

成立,而且白熊那組是反過來的。

bear 這組:沒提到白熊時,J-space 裡幾乎沒有它(max p = 0.0001)。叫它想白熊,升到 0.0075。叫它不要想白熊,升到 **0.1079**,比叫它想的時候高一個數量級以上。抑制的指令沒有把概念壓下去,反而把它推進 workspace。這就是白熊效應,論文圖 10 有跨 model 的版本。

lemon 這組是乾淨的正向。指令說的是 citrus fruit, **lemon** 從沒出現在 prompt 裡, J-space 讀到 0.0114;把指令換成想呼吸,掉回 0.0001,跟完全沒有指令一樣。動的是被指定的那個概念,不是「有指令就整體抬高」。

絕對值都很小(0.0001 到 0.1),要看的是同一個 target 在不同指令之間的比值。

7.2 把概念消除

交換是把座標換過去,消除是把座標歸零。同一套機制, $\sigma(c) = 0$:

$$V = [v_1 \cdots v_k], \quad h \leftarrow h - \alpha V (V^\dagger h)$$

一次投掉整組方向,不是一個一個減。J-lens 向量彼此不正交,一個一個減會重複扣掉共用的成分。

從最小的介入開始加碼:一個方向、一個位置 → 一個方向、所有位置 → 整組 {France, French, Paris}、所有位置,另外也試直接把答案方向 Paris 拿掉。

```
In [20]: def ablate_span(words, alpha=1.0):
        """h <- h - alpha * V (V+ h): project out the span of several J-lens
        directions."""
        def f(x, l):
            V = mx.stack([jlens_vec(tid(w), l) for w in words], axis=1)
            return x - alpha * (V @ (mx.linalg.pinv(V, stream=mx.cpu) @ x))
        return f

        PROMPT = "The capital of France is"
        ids = encode(PROMPT)
        fpos, last = where(ids, "France"), ids.shape[1] - 1
        WIDE = [l for l in Jlens if 20 <= l < 62]
        ALL = list(range(ids.shape[1]))

        RUNS = [
            ("baseline", (), BAND),
            ("France @France L40-55", ((fpos, ablate_span(["
France"])),), BAND),
            ("France @France L20-61", ((fpos, ablate_span(["
France"])),), WIDE),
            ("France @every pos L40-55", tuple((p, ablate_span(["
France"])), for p in ALL), BAND),
            ("France @every pos L20-61", tuple((p, ablate_span(["
France"])), for p in ALL), WIDE),
            ("Paris @last pos L40-55", ((last, ablate_span(["
Paris"])),), BAND),
            ("Paris @last pos L20-61", ((last, ablate_span(["
Paris"])),), WIDE),
        ]

        print(f"{PROMPT!r}")
        for tag, edits, layers in RUNS:
            sc = edited(ids, edits, layers)
            print(f" {tag} {top1(sc)!r:9s} {probs(sc, ('Paris', ('
Paris',)))}")
```

'The capital of France is'

baseline	' Paris'	p(Paris)=0.617
-France @France L40-55	' Paris'	p(Paris)=0.524
-France @France L20-61	' Paris'	p(Paris)=0.559
-France @every pos L40-55	' Paris'	p(Paris)=0.197
-{France,French,Paris} @every	' not'	p(Paris)=0.001
-Paris @last pos L40-55	' not'	p(Paris)=0.040
-Paris @last pos L20-61	' not'	p(Paris)=0.000

消除成不成,看你動多少。

- 只投掉 `France` 一個方向,只在 `France` 那個位置: `p(Paris)` 從 0.617 到 0.524,幾乎沒事。
- 同一個方向,層數放寬到 L20-61:0.559,也一樣。
- 一個方向,但每個位置都投:0.197,掉了三分之二。
- 整組 `{France, French, Paris}`、每個位置:0.001,top-1 變成 `' not '`。
- 直接投掉答案方向 `Paris`,只在最後一個位置:0.040;層數放寬到 L20-61 是 0.000。

兩件事看得出來。一是概念不是只存在提到它的那個位置上, `France` 的資訊在 forward 的過程中 已經散到別的位置,只掐一個點沒用。二是要投整組,一個一個減會漏掉共用的成分。

7.3 選擇性:明確報告 vs 自動處理

論文的說法是:進了 workspace 的內容,只有在需要「報告」或「彈性使用」時才起作用;自動處理那條路走不到它(原文圖 8 的例子就是一段西班牙文)。

兩個 prompt 共用一句西班牙文,介入完全相同,把西班牙文那幾個 token 的 `Spanish` 座標換成 `French` :

- **明確報告**:問這句是什麼語言。翻得動,代表這個概念被拿去回答問題。
- **自動處理**:叫它照原樣接下去。如果論文對,這條應該不受影響,照樣接西班牙文。

關鍵在強度要掃過去,不是挑一個 α 交差。

```
In [21]: def swap_lang(alpha, src=" Spanish", tgt=" French"):  
          return lambda x, l: swap(x, jlens_vec(tid(src), l),  
                                   jlens_vec(tid(tgt), l), alpha)  
  
EXPLICIT = 'Sentence: "El sol se pone en el oeste." That sentence is  
written in'  
AUTO = 'Continue in the same language: "El sol se pone en el'  
  
for prompt, pairs, lastw in [  
    (EXPLICIT, (("Spanish", (" Spanish",)), ("French", (" French",))),  
    " oeste"),  
    (AUTO, (("oeste", (" oeste",)), ("ouest", (" ouest",))), " el")]:  
    ids = encode(prompt)  
    toks = [tokenizer.decode([int(t)]) for t in ids[0].tolist()]  
    a = where(ids, "El")  
    b = max(i for i, t in enumerate(toks) if t == lastw)  
    ps = list(range(a, b + 1))  
    print(f"{prompt!r}\n    patching positions {ps[0]}..{ps[-1]}")  
    for alpha in (0, 0.5, 1.0, 1.5, 2.0, 3.0):  
        edits = () if alpha == 0 else tuple((p, swap_lang(alpha)) for p in  
ps)  
        sc = edited(ids, edits)  
        print(f"    alpha={alpha:<4} {probs(sc, *pairs)}    top1=  
{top1(sc)!r}")
```

```
'Sentence: "El sol se pone en el oeste." That sentence is written in'  
patching positions 3..9  
alpha=0    p(Spanish)=0.108  p(French)=0.006    top1=' which'
```

```

alpha=0.5  p(Spanish)=0.103  p(French)=0.014  top1=' which'
alpha=1.0  p(Spanish)=0.099  p(French)=0.015  top1=' which'
alpha=1.5  p(Spanish)=0.003  p(French)=0.131  top1=' which'
alpha=2.0  p(Spanish)=0.002  p(French)=0.123  top1=' which'
alpha=3.0  p(Spanish)=0.002  p(French)=0.119  top1=' which'
'Continue in the same language: "El sol se pone en el'
  patching positions 7..12
alpha=0    p(oeste)=0.014  p(ouest)=0.000  top1=' horizonte'
alpha=0.5  p(oeste)=0.015  p(ouest)=0.000  top1=' horizonte'
alpha=1.0  p(oeste)=0.017  p(ouest)=0.000  top1=' horizonte'
alpha=1.5  p(oeste)=0.000  p(ouest)=0.000  top1=' Spanish'
alpha=2.0  p(oeste)=0.000  p(ouest)=0.000  top1=' Spanish'
alpha=3.0  p(oeste)=0.000  p(ouest)=0.000  top1=' Spanish'

```

一半成立。

明確報告那條翻得動,但要 $\alpha = 1.5$ 。 $\alpha \leq 1.0$ 時 `p(Spanish)` 還是 0.099、
`p(French)` 0.015;到 $\alpha = 1.5$, `p(Spanish)` 掉到 0.003, `p(French)` 升到 0.131。

自動接續那條在 $\alpha \leq 1.0$ 完全不受影響,照樣接 `' horizonte'`, `p(oeste)` 甚至微升。
 到 $\alpha = 1.5$ 就壞了,但不是變成法文:top-1 變成 `' Spanish'`,它開始講語言的名字,不接句子了。

所以拿不到論文那個乾淨的分離。兩條路的門檻是同一個:能讓明確報告翻過去的強度,剛好也是讓自動接續脫軌的強度。能說的只有下半句,在自動接續還正常的強度下,概念的改動確實沒有影響它。

7.4 口頭報告:注入一個概念,問它在想什麼

論文最戲劇化的一項:把一個概念寫進 residual stream,再問它腦裡有什麼,它會說出那個字。

這裡要小心的是,「它說出那個字」有兩種解釋:一種是它讀到了自己 workspace 裡的內容 再報告出來,另一種是注入直接偏移了輸出分布,不管問什麼它都會說。所以要有對照組:同一個注入,換一個完全無關的問題。如果那邊也照樣說出來,就不能叫報告。

```
In [22]: ASK = "What single word is on your mind right now? Answer with one word:"
CONTROL = "The capital of France is"

def inject(word, alpha):
    def f(x, l):
        v = jlens_vec(tid(word), l)
        return x + alpha * v / mx.linalg.norm(v) * mx.linalg.norm(x)
    return f

def generate(ids, n=5, edits=(), layers=BAND):
    """Greedy continuation, re-running the whole forward for each token."""
    cur, out = ids, []
    for _ in range(n):
        t = int(mx.argmax(edited(cur, edits, layers)))
        if t in tokenizer.all_special_ids:
            break
        out.append(t)
        cur = mx.concatenate([cur, mx.array([[t]])], axis=1)
    return tokenizer.decode(out)

for word, alpha in (("volcano", 0.0), ("volcano", 0.5), ("volcano", 1.0),
                    ("guitar", 0.5)):
    line = []
    for prompt, n in ((ASK, 5), (CONTROL, 4)):
        ids = encode(prompt)
        edits = () if alpha == 0 else tuple((p, inject(word, alpha))
                                           for p in range(ids.shape[1]))
        line.append(generate(ids, n, edits))
    print(f"inject{word:9s} alpha={alpha:<4} ask -> {line[0]!r:26s}"
          f" control -> {line[1]!r}")
```

```
inject volcano alpha=0.0 ask -> "\n\n<think>\nHere's"
control -> ' Paris.\n\n<think>'
inject volcano alpha=0.5 ask -> ' volcano.\n\n<think>\n'
control -> ' volcano.\n\n<think>'
inject volcano alpha=1.0 ask -> ' volcano.\n\n<think>\n'
control -> ' volcano, and the'
inject guitar alpha=0.5 ask -> ' guitar.\n\n<think>\n'
control -> ' guitar.\n\n<think>'
```

不成立,而且是對照組把它打掉的。

注入之後它確實會說。問「你現在想到哪一個字」, $\alpha = 0.5$ 就答 ' volcano.', 換成 guitar 就答 ' guitar.' ;沒注入的時候它只會進 <think> 。

但對照組也照樣說。同一個注入,問 The capital of France is ,答的是 volcano 或 guitar, 不是 Paris。 α 從 0.5 到 1.0 都一樣。

所以在這顆 model 上,這個注入不能叫「報告 workspace 裡的內容」,它就是把輸出分布推向那個字而已。要分開這兩件事,得找到一個強度或位置,讓「問它想什麼」會說出來、「問別的問題」不會,我掃過的範圍裡沒有。

7.5 內部推理:交換之後,它會拿新概念去算嗎

最難的一項。交換之後 model 要拿新概念去**推**,而不是把它說出來。

兩種題型:

數腳(論文圖 6 的例子)。把 spider 換成 ant,問腳的數量,答案該從 8 變 6。8 和 6 都不在 prompt 裡,也不是被交換的那兩個字。這顆 model 是 thinking model, 問句形式會讓它先吐 \n\n,所以用完成句的寫法。

兩跳事實。The capital of the country where the Eiffel Tower stands is → Paris。中間那一跳 France **完全不在 prompt 裡**,是 model 自己算出來的。把 France 換成 China, 如果它答 Beijing,那它是拿新概念又跑了一次第二跳;如果它答 China,那只是把注入的東西 講出來。這個題型能把「推理」和「洩漏」分開。

第二種還要挑位置。France 不在輸入裡,所以沒有「France 那個 token 的位置」可以改, 只能改別的位置。三個選擇都試:最後一個位置、Tower 那個位置、除了最後一個以外全改。

```

In [23]: EIGHT, SIX = ("8", " eight"), ("6", " six")

print("legs: swap the animal at the animal's own position")
for prompt, src, tgt, keep, want in [
    ("A spider has", " spider", " ant", EIGHT, SIX),
    ("The number of legs on a spider is", " spider", " ant", EIGHT,
SIX)]:
    ids = encode(prompt)
    pos = where(ids, src.strip())
    print(f" {prompt!r} swap{src} ->{tgt}")
    for alpha in (0, 2.0, 3.0):
        edits = () if alpha == 0 else (
            (pos, lambda x, l, a=alpha: swap(x, jlens_vec(tid(src), l),
                jlens_vec(tid(tgt), l), a)),)
        )
        sc = edited(ids, edits)
        print(f" alpha={alpha:<4} {probs(sc, (keep[0], keep), (want[0],
want))}")
        f" top1={top1(sc)!r}")

TWO_HOP = "The capital of the country where the Eiffel Tower stands is"
ids = encode(TWO_HOP)
last = ids.shape[1] - 1
TARGETS = [
    ("last position only", [last]),
    ("the Tower position", [where(ids, "Tower")]),
    ("every position but last", list(range(1, last)))]

def swap_fc(alpha):
    return lambda x, l: swap(x, jlens_vec(tid(" France"), l),
        jlens_vec(tid(" China"), l), alpha)

print(f"\ntwo-hop: {TWO_HOP!r}")
print(f" 'France' appears in the prompt: {'France' in TWO_HOP}")
for tag, ps in TARGETS:
    print(f" patch {tag} {ps}")
    for alpha in (0, 1.0, 2.0, 3.0):
        edits = () if alpha == 0 else tuple((p, swap_fc(alpha)) for p in
ps)
        sc = edited(ids, edits)
        print(f" alpha={alpha:<4} "
            f"{probs(sc, ('Paris', (' Paris',)), ('Beijing', ('
Beijing',)), ('France', (' France',)), ('China', (' China',)))}"
            f" top1={top1(sc)!r}")

```

legs: swap the animal at the animal's own position

'A spider has' swap spider -> ant

alpha=0	p(8)=0.124	p(6)=0.004	top1=' '
alpha=2.0	p(8)=0.092	p(6)=0.003	top1=' '
alpha=3.0	p(8)=0.083	p(6)=0.003	top1=' '

'The number of legs on a spider is' swap spider -> ant

alpha=0	p(8)=0.050	p(6)=0.009	top1=' '
alpha=2.0	p(8)=0.008	p(6)=0.007	top1=' '
alpha=3.0	p(8)=0.008	p(6)=0.007	top1=' '

two-hop: 'The capital of the country where the Eiffel Tower stands is'

```

'France' appears in the prompt: False
patch last position only      [11]
    alpha=0    p(Paris)=0.394  p(Beijing)=0.000  p(France)=0.001
p(China)=0.000  top1=' Paris'
    alpha=1.0  p(Paris)=0.262  p(Beijing)=0.003  p(France)=0.000
p(China)=0.000  top1=' Paris'
    alpha=2.0  p(Paris)=0.000  p(Beijing)=0.000  p(France)=0.975
p(China)=0.000  top1=' France'
    alpha=3.0  p(Paris)=0.000  p(Beijing)=0.000  p(France)=0.980
p(China)=0.000  top1=' France'
patch the Tower position      [9]
    alpha=0    p(Paris)=0.394  p(Beijing)=0.000  p(France)=0.001
p(China)=0.000  top1=' Paris'
    alpha=1.0  p(Paris)=0.373  p(Beijing)=0.000  p(France)=0.001
p(China)=0.000  top1=' Paris'
    alpha=2.0  p(Paris)=0.148  p(Beijing)=0.011  p(France)=0.000
p(China)=0.000  top1=' Paris'
    alpha=3.0  p(Paris)=0.163  p(Beijing)=0.010  p(France)=0.000
p(China)=0.000  top1=' Paris'
patch every position but last [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    alpha=0    p(Paris)=0.394  p(Beijing)=0.000  p(France)=0.001
p(China)=0.000  top1=' Paris'
    alpha=1.0  p(Paris)=0.350  p(Beijing)=0.000  p(France)=0.001
p(China)=0.000  top1=' Paris'
    alpha=2.0  p(Paris)=0.000  p(Beijing)=0.189  p(France)=0.000
p(China)=0.000  top1=' Beijing'
    alpha=3.0  p(Paris)=0.000  p(Beijing)=0.218  p(France)=0.000
p(China)=0.001  top1=' Beijing'

```

數腳那組不成立。p(8) 從 0.124 掉到 0.083, p(6) 從 0.004 完全沒動。交換把舊概念弄壞了,沒把新概念的後果裝上去。

兩跳事實那組成立,但位置決定一切:

- 只改最後一個位置: $\alpha = 2$ 時 p(France) 衝到 0.975, top-1 是 ' France'。它把被交換的概念直接說出來,這就是前面一直提的 late-layer 洩漏,現在看得到數字。
- 只改 **Tower** 那個位置:幾乎沒反應, p(Beijing) 最多 0.011。單一個位置扛不起來。
- 除了最後一個以外全改: $\alpha = 2$ 時 top-1 變成 ' Beijing', p(Paris) 掉到 0.000, 而 p(France)、p(China) 都還在 0.000 到 0.001。

最後那個就是要找的東西。它沒有把 China 講出來,講的是**第二跳的結果**;被換掉的概念從頭到尾不在輸入裡,所以也不可能是 input-copying。

兩個題型合起來,結論跟 Neel Nanda 團隊的複現一致:多跳事實的替換站得住,需要「算」的那種不行。數腳要的是「知道 ant 有 6 隻腳」這種屬性查詢,兩跳事實要的是「拿 China 再跑一次首都查詢」,後者這顆 model 做得到。

六項跑完的樣子

面向	這顆 model(Qwen3.6-27B-4bit)
flexible generalization	成立。一個介入,四個任務一起改(\$6)
directed modulation	成立。叫它不要想白熊,白熊反而更清楚
internal reasoning	成立,但只有兩跳事實那種,數腳那種不行
概念消除(論文驗因果的手法)	成立,但要投整組方向、每個位置都投
selectivity	一半。明確報告翻得動,但拿不到跟自動處理的乾淨分離
verbal report	不成立。注入之後,問什麼都會說出那個字

四項成立、一項一半、一項不成立。這是一顆 27B 的 4-bit 量化 model,不是 Sonnet 4.5, 效應本來就該弱得多。

有一個共同的規律值得記:成立的那幾項都要改在內容位置,而且往往要改一整片位置。只改一個點不夠,改最後一個位置就會變成把概念直接說出來。

小結

- **residual stream** 是所有層共用的座標系,所以才有得讀。
- **logit lens** 把最後那顆頭借來讀中間層,便宜,但暗中假設 $J_\ell = I$ 。
- **Jacobian lens** 把 J_ℓ 真的算出來,讀同一條 residual 卻早好幾層看到答案。非英文問句差最多,中文和法文早了 30 幾層。
- 從零算 J_ℓ 只需要 `mx.vjp` 加兩個實務 workaround,一條 prompt 的結果就跟一千條擬合的公開 lens 大致同向。
- **J-space** 是稀疏非負組合。25 個方向只解釋 residual 的 1.5% 到 8%, 但比隨機挑的高一到兩個數量級(\$5)。
- **改得動**:同一個介入把 `France` 換成 `China`,四個不同的問題一起改口; 把整組方向投掉,答案就消失。
- 六個面向裡四個成立、一個一半、一個不成立,而且成不成很吃你改在哪些位置(\$7)。

該保留的懷疑

- **線性近似**。 J_ℓ 是一階展開,後面幾十層並不是線性的。
- **全域先驗**。公開 lens 是跨一千條 prompt 平均的,會帶進跟當前輸入無關的偏好。
- **input-copying**。如果要探測的字本來就出現在 prompt 裡,lens 會在那個位置直接把它讀成第一名,那反映的是輸入,不是「內部想法」。上面的例子沒有踩到這點(`Paris` 不在 prompt 裡),但要自己延伸實驗時得檢查(issue #5)。
- **名字取得不好**。「J-space」並不是 residual stream 的一個線性子空間。照論文自己的 methods,它是「在稀疏限制下的一堆多面錐的聯集」。
- **循環論證的疑慮**。一個依「對最終輸出的影響」定義出來的空間,本來就會跟最終輸出高度相關。
- **不是每個實驗都能重現**。獨立重現裡,多步事實的替換站得住,但詩韻規劃和心算沒有重現出來 (Nanda 團隊的 review、[tao-hpu/jspace-replication](#))。

參考資料

- [anthropics/jacobian-lens](#) 官方實作
- [neuronpedia/jacobian-lens](#) 擬合好的 lens
- [Interpreting GPT: the logit lens](#)
- [A Mathematical Framework for Transformer Circuits](#)
- [On the Biology of a Large Language Model](#)
- [J-Space: Yet Another LLM Mind Reader?](#)
- [A Review of Anthropic's Global Workspace Paper](#)
- [Gated Delta Networks](#)